

Music Room

Music, Collaboration and mobility

Summary: *Creation of a complete mobile solution focused on music and user experience.*

Version: 6

Contents

I Preamble **2**
I.1 The musical experience . . . 2

II AI Instructions **3**

III Objectives **5**

IV General instructions **6**
IV.1 Instructions concerning the software architecture . . . 6
IV.2 Instructions concerning the mobile experience . . . 6

V Mandatory part **7**
V.1 User . . . 7
V.2 Services . . . 8
V.2.1 Music Track Vote . . . 8
V.2.2 Music Control Delegation . . . 9
V.2.3 Music Playlist Editor . . . 9
V.3 Server . . . 10
V.4 API . . . 10
V.5 Mobile application . . . 10
V.6 Securing . . . 11
V.7 Ramp-up . . . 11
V.8 Agility, quality and continued integration . . . 11

VI Bonus part **12**
VI.1 Multi-platform support . . . 12
VI.2 Reflection on the IoT . . . 12
VI.3 Free subscription vs. Paid subscription . . . 12
VI.4 Offline Mode . . . 12

VII Submission and peer-evaluation **14**

Chapter I

Preamble

I.1 The musical experience

The musical "consumption" is a personal experience everyone lives in their own personal way:

- Surrounding with music everyday,
- Enjoying music when sharing time with friends,
- Using music to isolate oneself or focus,
- Only believing in performance, live shows or festivals.

Chapter II

AI Instructions

🟢 Context

AI is now a powerful coding partner — alongside your peers — for tackling large and demanding projects. You will guide it through both technical and non-technical aspects of your work.

AI tools can boost your efficiency and improve the quality of your output, but you should be able to dive deep into any part of the project without relying on them.

Your AI partner supports you, but you remain fully responsible for making informed technical decisions and to clearly explain and defend them.

🟢 Main message

- 👉 Strive for a mature and responsible use of AI.
- 👉 Never let AI take responsibility for decisions — especially when it lacks awareness of your goals, constraints, or team dynamics.
- 👉 Maintain creativity, innovation, and human oversight through active collaboration with your peers. AI is trained on existing data and rarely generates truly new ideas.
- 👉 Stay informed about emerging trends and be ready to adapt to new concepts and technologies.

🟢 Learner rules:

- Maintain intellectual leadership over your projects and make your own informed decisions.
- Prioritise the collective intelligence of your team and peers.
- Actively stay informed about the ongoing evolution of AI technologies.

Phase outcomes:

- AI engineering skills.
- Increased efficiency.
- Greater reliability and quality.
- A pioneering mindset.

Comments and examples:

- Your peers can identify trade-offs, question assumptions, and help you improve. The first answer from an AI might not be the best — it may lack efficiency, security, or real added value. Now more than ever, you should rely on your peers.
- AI can make you faster, but your peers make you better. Collaboration, discussion, and mutual challenge are key to success.
- Be transparent about how AI was used in your projects, and clearly identify what was generated by AI tools.

✓ Good practice:

> I asked AI to help generate unit tests for my API. I reviewed them with my teammate, and we adjusted them for edge cases. It saved time, and we both learned something new.

✗ Bad practice:

> I had AI generate the entire architecture of my project. It "works," but when I'm asked to explain design decisions during the peer review or in front from of a customer, I cannot. I lose credibility and I fail.

Chapter III

Objectives

This project aims to tackle all the concepts necessary to the creation of a mobile, connected and collaborative application taking into account the constraints of a real product.

You will have to work on the client/server architecture of you solution, take decisions for you data storage, define an API that will be used as a communication channel between your server and your clients, anticipate the problems of ramp-up and securing, and learn to work with third parties, integrating external SDKs.

There are a lot of subjects. You should dispatch the different tasks between your project team members.

The following services will have to be implemented:

- **Music Track Vote** - Live music chain with vote,
- **Music Control Delegation** - Music control delegation,
- **Music Playlist Editor** - Real time multi-user playlist edition



`Warning the SDK you choose must not do your work.`

Chapter IV

General instructions

IV.1 Instructions concerning the software architecture

There is no constraint as to strategies or language you must use for the back-end: you will have to evaluate the benefits and disadvantages of the different available technologies and identify the ones that will suit the project best. You will have to justify your decisions.

You cannot commit libraries you would not have written yourself in your repo. Your project's dependencies must be automatically downloadable from a clone of your repo thanks to a Makefile or similar mechanisms.

IV.2 Instructions concerning the mobile experience

The mobile application must allow the user to perform all necessary actions related to the project. It can be implemented either for **Android** or **iOS**, using the technology of your choice.

Chapter V

Mandatory part

V.1 User

When the user runs the application for the first time, they must create an account on the application. They will choose between a mail/password or a social network account (**Facebook** or **Google**) registration.

Once the user has an account on your platform, the application must allow them to link their network account (**Facebook** or **Google**).

In their profile, the user must be able to state and update:

- their public informations,
- informations only available to their friends,
- their private informations,
- their music preferences.



If the user has created an account with a mail/password, the application must demand a mail validation and invites the user to change their password if they forgot it.

~~V.2 Services~~

~~On the application, user must access at least 2 functions out of the following 3 ones: **Music Track Vote**, **Music Playlist Editor** and **Music Control Delegation**.~~

V.2.1 Music Track Vote



`Live music chain with vote.`

Some people are gathered in a place (party, event...). Your service must allow anyone to suggest or vote for the next track coming in the current playlist. If a track gets many votes, it goes up in the list and is played earlier.

Visibility management (Public/Private) must be integrated to the service:

- By default, your event is public.
- All the users can find your event and vote if your event is public.
- Invited users are the only ones who can find the event and vote if your event is private.

A license management must be integrated to the service:

- By default, everyone can vote.
- With the right license, the invited people are the only one who can vote.
- With the right license, the people located in a specific place for at a specific time (between 4 and 6PM for instance) will be able to vote.



`You should especially care about the management of competition problematics: for instance, if several people vote for different tracks or the same one in a playlist.`

~~V.2.2 Music Control Delegation~~



~~Music control delegation.~~

~~A license management must be integrated to the service. It must be specific for each device attached to the user's account. The user can choose to give the music control to different friends.~~

~~V.2.3 Music Playlist Editor~~



~~Real time multi-user playlist~~

~~Collaborate with your friends or people with the same musical tastes to create playlists in real-time. This way, users can create original radio stations.~~

~~A visibility management (Public/Private) must be implemented to the service:~~

- ~~By default, a playlist is public.~~
- ~~If it is public, every user have access to the playlist.~~
- ~~If it is private, only the invited users can have access to the playlist.~~

~~A license management must be implemented to the service:~~

- ~~By default, everyone can edit the playlist.~~
- ~~With the right license, the invited users are the only ones who can edit the playlist.~~



~~You should especially care about the management of competition problems: for instance, if several people move different tracks or the same one in a playlist.~~

V.3 Server

All the service data will be stored on the back-end side. The back-end is the reference. It is the keeper and representative of the "truth". You can use the technology and frameworks your like (php, nodejs, golang, firebase, etc.).

V.4 API

The API will be the access point to your back-end for all the applications. Some developers will lean on your API for various context integrations. Hence, the documentation regarding this API is essential and you are the first developers who will use it. This API reference documentation must introduce methods, inputs and outputs. It can be self-generated with Swagger, for instance.

You should create an API that endorses the principle of REST but there are others out there (new trends spawn on a daily basis). Anyway, you will have to be able to justify your choice and explain its characteristics.

You should endorse JSON for your exchange format with the API but there are others out there. Anyway, you will have to be able to justify your choice and explain its characteristics.

V.5 Mobile application

Applications must only be "remote control" to the back-end and the back-end's address must be configurable on the application for tests.

Authentication support through a social network like (**Facebook** or **Google**) must be implemented in the mobile application. Resources are available on: developers.facebook.com and developers.google.com.

V.6 Securing

~~An authenticated user on your solution must have access to their data, but not to other users' data. You must plan malicious behaviors (bruteforce of your API, session theft, etc.) but your API is not expected to be unbreakable:~~

- ~~you must implement mechanisms to protect your users,~~
- ~~you must identify other hazards and explain the practicable protections.~~

~~Any action on the mobile application must generate logs on the back-end.~~

- ~~Platform (Android, iOS, etc.),~~
- ~~Device (iPhone 6G, iPad Air, Samsung Edge, etc.),~~
- ~~Application Version.~~

V.7 Ramp-up

You must be able to evaluate the load your API and your back-end can support, that is, justify and measure the number of users that can simultaneously use your 3 services. You can use AB (Apache Benchmark), Gatling, Siege, Tsung, JMeter for instance.

Don't forget to specify the servers characteristics (CPU, RAM, Cloud or Premise, etc.). The maximum number of users should be consistent with the platform choice. Dozens for a Raspberry, thousands for a low-end server.

~~V.8 Agility, quality and continued integration~~

~~You have a lot of layers and functionalities to implement for this project, and you will have to work in a team. You should arrange yourself to show agility, to be able to question your own decisions, and to set specific one-off tests for each layer~~.



For obvious security reasons, any credentials, API keys, env variables etc... must be saved locally in a .env file and ignored by git. Publicly stored credentials will lead you directly to a failure of the project.

Chapter VI

Bonus part

The bonuses proposed here match real business problematics you will face when developing a mobile application.

VI.1 Multi-platform support

Once you have a server and a functional API, you can make your services available on various platforms. You should already support a mobile platform (Android or iOS). As a bonus, you could also make your service web "responsive" so it can adapt to any screen size.

According to your technological choices, web support can require that you rewrite your client code entirely or almost entirely.

VI.2 Reflection on the IoT

You can implement a mechanism such as IBeacon on your event. For instance, when people get near a public event registered on your service, they automatically receive informations about the event, how to access it, the kind of music etc.

VI.3 Free subscription vs. Paid subscription

Nowadays, web solutions and mobile applications often offer the user to choose between a free limited subscription and a several unlimited paid ones.

This bonus will allow you to implement this kind of logic in your application. You will then have to allow users to switch between two offers you will have established beforehand. Some of your application's functionalities (**Music Playlist Editor** for instance) will only be available to users with a paid subscription.

VI.4 Offline Mode

With this bonus, you will implement a mechanism that allows the users to enjoy the application offline, when, for instance, when it won't have mobile service. In that case,

the experience and functions proposed by the application might be completely different.

If the application is used offline, you should plan a synchronisation. Beware, though, sync mechanisms carry their lot of problems!:

- Managing conflicts and concurrency,
- Managing obsolete data on the mobile application.



The bonus part will only be assessed if the mandatory part is PERFECT. Perfect means the mandatory part has been integrally done and works without malfunctioning. If you have not passed ALL the mandatory requirements, your bonus part will not be evaluated at all.

Chapter VII

Submission and peer-evaluation

Turn in your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Don't hesitate to double check the names of your folders and files to ensure they are correct.